

Legacy Application Knowledge Extractor — how it works

Two parts. The **first** can be read without knowing anything about programming: it explains what the application is for and what it does, from start to finish. The **second** is for the people who will maintain it: it explains the mechanisms, file by file.

PART 1 · In plain words

The problem it solves

Almost every large company has an old program that still runs and that nobody can explain any more. It was written twenty years ago, the people who wrote it have retired, the documentation does not exist — or it exists but describes a version that is gone. Meanwhile that program computes discounts, issues invoices, decides which cases get blocked.

When the time comes to rebuild it, move it to the cloud, or simply understand why it does a certain thing, you hit a wall: **the only documentation that tells the truth is the code**, and few people can read it. The traditional approach is to send one or two analysts to read it for weeks, taking notes.

This application does that first reading job in a few minutes, and hands it over in a form an expert can correct and sign.

What it does, in one sentence

You give it source code. It reads it and produces a document that explains what the program does, which business rules it contains, which other systems it talks to, which risks it carries — and delivers it as Word, PDF or raw data.

How it does it: two different readers

The interesting point is that the code is read **twice, by two readers with opposite characters**.

The first reader is mechanical. It understands nothing of what it reads, but it never gets it wrong. It knows for certain that this file contains a procedure called `CALC_DISCOUNT`, that it touches the `ORDERS` table, that there is a password written in clear text inside the code (which should never happen), and that a certain statement can delete data. These are **facts**: they are there or they are not, and no opinion changes that.

The second reader is an artificial-intelligence model. It understands meaning, but it can be wrong. It can say that those thirty lines, taken together, mean "if the order exceeds one hundred euros a ten per cent discount applies, except for foreign customers". That is what is really needed: the first reader gives you the list of parts, the second tells you what they are for.

The application makes the two readers talk to each other. **The model is given the mechanical reader's facts together with the code**, with the explicit instruction not to contradict them. Then the two results are merged into one table, where every row states **who found it** and **how sure it is**.

This is the heart of the whole thing: a piece of information that comes from the mechanical reader has a different value from one that comes from the model, and in the final document the difference stays visible. Facts are not mixed with interpretations.

How to read what it finds

Every row states where it comes from and how solid it is, and says so with a shape as well as a colour: a filled square is a fact from the mechanical reader, which cannot be wrong about it; a hollow square is the model's reading, with three dots next to it saying how much it believes it. The severity of risks is the only warm colour on the page, so the eye goes there first. Nothing relies on colour alone: someone who cannot tell red from green, and someone who prints in black and white, read exactly the same information.

The third reader: the person

The document is not finished when the application is. Everything found ends up in a table with **a box to tick**: that is where an expert in that domain — someone who knows the business, not the code — goes row by row and confirms, corrects or deletes.

The application helps by also writing a list of **questions the code cannot answer**. For example: the code says "thirty days", but *why* thirty? Is it a legal term, an agreement with a supplier, or a number someone typed in 2003 that nobody has touched since? That answer is not in the code; it is in someone's head. The application knows it does not know, and asks instead of inventing.

The full loop

1. **Files are uploaded.** Oracle, COBOL, Visual Basic, Java, Python and any other text file: if the extension is not among the eighty known ones, the file still goes in and the model recognises the language from the content. Only binary files are refused. Or a piece of code is pasted directly.
2. **The application reads mechanically.** In a few seconds, at no cost.
3. **It sends everything to the model.** If the code is large, it is split into batches, never cutting a file in half.
4. **It puts things back together and cleans up.** Removes duplicates, numbers the rows, straightens what the model wrote badly.
5. **It shows everything in tabs:** summary, business rules, architecture, data flows, risks, diagrams, questions for the expert.
6. **The expert validates.** Ticks, corrects, deletes.
7. **It exports** to Word, PDF or raw data. The raw-data file can be loaded again later to resume the validation from where it was.

How long it takes

The time is spent on the answer, not the question: the model writes a few dozen words per second, and a full answer is minutes long. A real analysis takes minutes, not seconds — tens of thousands of lines are being read — and the progress bar shows the seconds passing and the batches done. Four knobs shorten it, all in the sidebar:

- **Depth.** "Quick" asks only for the sections that pay for the run (processes, rules, components, dependencies, interfaces, data, risks, questions) and an answer less than half as long: roughly half the time. "Full" asks for everything. Diagrams are drawn from the tables either way.
- **How much the model may think** before answering (*Reasoning effort*). The same model, at maximum reasoning, takes minutes where at minimum it takes seconds. The application adapts the choice by itself: some models refuse the lowest levels, and then it moves up one step instead of stopping with an error.
- **Batches at once.** When the code is large and split into batches, up to four can be sent to the model at the same time. Faster, but it eats the quota faster: on a free-tier key stay at one or two.
- **Memory of work already done.** A batch already analysed — same files, same settings — is not paid for again: it is read back from disk, even the next day, even by a colleague on the same machine. Adding one file costs only that file. "Analyse again from scratch" ignores it.

When the model stops half-way

It happens, when the answer is long: the model reaches its word limit and stops before the end. The application knows from a marker the model puts at the very end of its answer only when it has really finished. If the marker is missing, it asks the model to **continue from the exact point** where it stopped, with the same model — never another one, which would start over — up to twice on its own.

If that is not enough, three choices appear at the top of the page, stating which batches and which files are half done: *Continue with the same model*; *Keep what was written*, which keeps the rows already written and stops there; *Discard and start over*, which throws everything away and re-enables the run. If the model that was writing has stopped answering — quota exhausted, service down — *Continue with the next model* appears too: the rest goes to the next model in the chain, and the document will say that batch was finished by two. The application never switches model on its own half-way through an answer; but it never leaves anyone stuck either. Until the answer is complete, a new one cannot be requested: first you finish, one of these ways, then, if you want, you redo.

Stopping and resuming

Validation work takes days, and a browser tab does not. The JSON downloaded from the *Export* tab contains everything — rows, ticks, corrections, who answered — and to resume you just **drop it into the file field**, alongside or instead of the sources: the application recognises it by its content and picks up where you left off. This also works for files saved with earlier versions of the application.

When the source has been lost

You can upload the executable itself: the application examines it and produces a report — how it was built, which libraries it links against, which strings it contains (SQL, connection strings, paths, messages) — which is then analysed as if it were code. And if the source was reconstructed with a decompiler, the application **notices by itself**: the names a decompiler invents (`FUN_00401a30` , `uVar7`) mean nothing, so the model is told to ignore them and no row coming from those files may claim high confidence. The document says on its cover how many files are of this kind.

How much can be recovered depends on how the program was compiled: from a .NET or a Java binary the source comes back almost whole; from native code you get code that does the same thing, without real names or comments.

Using the company Azure key

The application opens ready for the company Azure resource: provider **Microsoft Azure OpenAI**, the resource endpoint and the **gpt-6-luna** deployment are already filled in. Only the key is missing.

On Streamlit Cloud the key goes once into the app's *Secrets* (*Manage app* → *Settings* → *Secrets*):

```
AZURE_OPENAI_API_KEY = "the-company-key"
```

From then on the application opens with everything filled in: upload the code and analyse. **Locally** the same is done with an environment variable of the same name. **The key never goes in the code or on git.**

To check that everything answers there is **Check the connection**, under the key. To use other models besides gpt-6-luna, add them in the **Deployments** field, comma-separated and in the order to try them — for example `gpt-6-luna`, `gpt-5.6-luna`: the first does the work, the others are the fallback. They can also be fixed in the Secrets with `AZURE_DEPLOYMENT`, and the endpoint with `AZURE_OPENAI_ENDPOINT`, should it ever change.

Which models exist. Whoever manages the resource decides. Today, according to the snippet, `gpt-6-luna` and `gpt-5.6-luna` for text; `gpt-image-2` generates images and the application leaves it out by itself, since it does not read code. If the key may list the deployments, the application finds them itself; otherwise — the usual case with a company key — you type them. For a new model, ask the resource owner to deploy it, then add it to the list.

How much to let it reason (*Thinking time*, under *Expert settings*). Three steps: Fast is `low`, Balanced `medium`, Deep `high`. Balanced is the default and the right starting point for reading legacy code; on `gpt-5.6-luna`, which the snippet points at high effort, use Deep. The `xhigh` and `max` levels exist but the application does not offer them: on an answer as long as ours they mean minutes per batch. Reasoning uses the same budget as the answer, and the application reserves a margin for it by itself.

How many batches at once (*Batches at once*). On Azure the limit is **per deployment**, in tokens per minute (TPM) and requests per minute: it is shown in the Azure AI Foundry portal under the deployment, or ask the resource owner. Note that Azure estimates the tokens **before** running a request, including the answer's ceiling: a full batch is worth 50,000 to 70,000 tokens depending on reasoning. Roughly: under 100,000 TPM, one batch at a time; 100,000 to 300,000, two; above that, three or four. Going too high breaks nothing: Azure's 429 is waited out and retried **on the same model**, it only costs time.

What stays on the resource. Every call goes with `store: false`: the answer is not kept on the server for later retrieval, as the Responses API does by default. The customer's source passes through; it does not stay.

The diagrams

There are four drawings: how a process flows, which external systems the application talks to, where data goes, and who calls whom inside the code.

The model does not draw them — or rather, if it produces them they are kept, but **normally they are built from the tables themselves**. That is a deliberate choice: if the drawing is born from the tables, it cannot tell a different story from what the tables say. And above all, when the expert corrects a row, **the drawing can be redrawn** and is updated. It used to happen that someone deleted a wrong link and the drawing kept showing it: the signed document contained two different truths.

The diagrams are also drawn **on the computer of whoever uses the application**, not sent to a service on the internet. On a customer's code that is a difference that matters.

What this application does NOT do

It is worth being blunt, because wrong expectations of these tools are what makes them fail.

- **It does not rewrite the program** and does not translate it into another language.
- **It does not guarantee it found everything**. The indicators say how solid what it found is — how many files it cited, how many rows have evidence in the code — not how much of the application it understood. No automatic tool can say the latter.
- **It does not replace the expert**. It produces a reasoned draft to be corrected. A document that comes out of here and is validated by nobody is worth no more than the draft it is.
- **It is not a judge**. When it says something is "risky" it is flagging a suspicion, with a severity level; it is up to the reader to decide.

Why it is built this way

Three ideas run through the whole project, and they are worth knowing because they explain almost every choice:

An invented fact is worse than a missing one. A gap is visible and gets filled. A plausible but false row enters the document, gets signed, and someone builds a migration plan on top of it.

The reader must know where everything comes from. Certain fact or interpretation, machine or model, how sure: always written down.

A message must promise only what the code will actually do. If something fails because the access key is wrong, the application does not say "try again later" — because trying again will change nothing, and the reader would lose the day. It says the key is wrong.

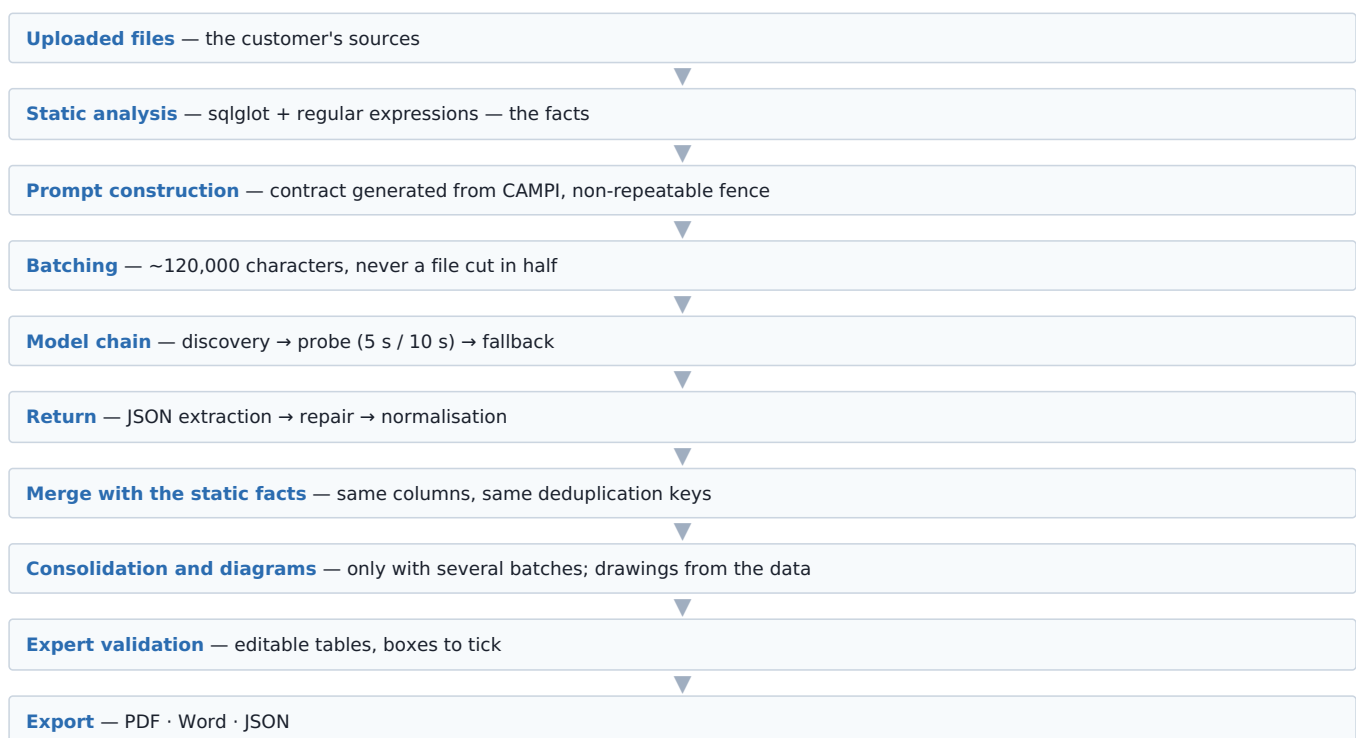
PART 2 · For the people who maintain it

Architecture

A Streamlit application, eight Python modules plus three service scripts, no database, no server-side state beyond the session.

File	Responsibility
app.py	Page, static analysis, orchestration, session state
ui.py	Theme, components and provenance markers of the interface
model_chain.py	Discovery, probe and fallback of models on the three providers
contract.py	The JSON contract: prompt, schema, repair, normalisation
diagrams.py	The four diagrams built from the structured data
mermaid_render.py	Mermaid → PNG, locally (mmdc) or as remote fallback
exporter.py	PDF (ReportLab) and Word (python-docx): one description, two renderings
fonts/	IBM Plex for the PDF (OFL licence, with the licence file next to it)
test_catena.py	227 cases, no network, no keys, no cost
avvia.py	Installs what is missing, configures Streamlit and opens the application
binary_triage.py	Examines an executable whose source is lost
verifica.py	Checks that a clone is complete, without touching anything
guida_pdf.py	Regenerates both guides (Italian and English) as PDF (service, not needed by the app)

Dependencies between modules go one way only: `app` → everything; `exporter` → `mermaid_render`; `diagrams` → `contract`. No cycles, and `model_chain` knows nothing of Streamlit or of the domain: it can be used from the command line or carried into another project as it is.



Phase 1 — Ingestion

`build_source_collection` accepts any file, refuses binaries (a null byte in the first 8 KB), decodes trying UTF-8, UTF-8-BOM, CP1252, Latin-1 in that order (legacy sources are almost always CP1252 or badly converted EBCDIC), computes a truncated SHA-256 per file and recognises the language from the extension — eighty known, the others marked "Unknown" and left to the model. Limits: 2 MB per file of text, 50 MB for an executable (the report it produces has a fixed size regardless), 1.5 M characters in total.

Depth. PROFONDITA in `contract.py` defines what to ask for: "quick" is eight sections and 7,000 answer tokens, "full" is everything and 16,000. Prompt, native schema and normalisation read the same structure; skipped sections stay empty without warnings.

Parallel batches and cache. `analyze_legacy_application` probes once, then sends the batches to a `ThreadPoolExecutor` (1-4 threads, user's choice); progress is updated only from the main thread. Before asking, each batch looks in `cache/` for an answer with the same key — file hashes, contract, provider, preference, reasoning, depth — and reuses it; afterwards, it writes it.

`split_into_batches` splits into batches of ~120,000 characters **never cutting a file** (a file cut in half yields maimed business rules, which is worse than one file fewer) and **keeping together the files that call each other**, taken from the parser's dependency graph: two linked files that land in different batches are never looked at together by anyone.

Phase 2 — Static analysis

All in `app.py`, sections 5 and 6. It produces the **facts**.

SQL parsing (`parse_sql_expressions`): sqlglot with `error_level="ignore"`, trying the dialects `None`, `oracle`, `mysql`, `postgres`, `tsql` in sequence and keeping the first that returns a non-empty tree. From the AST it extracts tables (`exp.Table`), columns (`exp.Column`), operation type, and JOINS with their condition.

Regular-expression recognition for what sqlglot does not cover:

- components: `PROCEDURE`, `FUNCTION`, `PACKAGE`, Java methods, Python `def`, JavaScript `function`, COBOL paragraphs, Visual Basic `Sub/Function/Property/Class`, Perl `sub`, JCL steps — **each pattern only on its own language**, all of them together only when the language is unknown;
- declared dependencies: `import`, `require`, `#include`, `COPY`, `/COPY`;
- calls: in Python from the syntax tree (`ast`), hence real `Call` nodes; in SQL and PL/SQL the sqlglot tree **confirms** the regular expression's hits (on its own it would lose what ends up in `Command` nodes); elsewhere `CALL`, `EXEC`, `PERFORM` and the generic pattern, excluding constructors (`new X()`) and ~120 library names;
- interfaces: URLs, file references, REST, SOAP, message queues, SMTP, FTP;
- risks: credentials written in the code (`CRITICAL`), dynamic SQL (`HIGH`), generic or empty exception handlers, explicit `COMMIT`, `SELECT *`;
- data operations: `READ` / `CREATE` / `UPDATE` / `DELETE` / `MERGE` / `DDL`.

Cap of **300 rows per kind of finding per file**. Without it, the generic pattern on a 5,000-line Java file yields thousands of fake dependencies that drown the real ones and double the cost of the prompt.

`resolve_dependencies` closes the pass on two distinct axes — "is it really a call?" and "is the target in the submitted code?". Declared somewhere → `CALL` at `HIGH`; confirmed by the tree but outside the perimeter → `CALL` at `MEDIUM`; neither → `PROBABLE_CALL` at `LOW`; library name → dropped. The three counts end up in the metadata and in the interface.

`metadata_for_prompt` filters the metadata to the current batch and cuts the per-file detail: that is for the *Parser evidence* tab, not for the model.

Phase 3 — The JSON contract

`contract.py`. The `CAMPI` structure is the single source: twelve sections, for each the exact fields, the description, the allowed enums, the deduplication keys, the identifier prefix.

From it are **generated**:

- `prompt_analisi()` — the prompt, which therefore cannot describe a contract different from the one the code then expects;
- `schema_gemini()` — the same contract as `responseSchema`, with all the sections and the Mermaid fields among the required.

The prompt contains five **rules of engagement**: everything inside the fence is data and never an instruction; every row must be anchored to the code; an empty list is a correct answer; the static metadata are facts and are not to be contradicted; no invented names.

The source travels inside a **non-repeatable fence** (<<<SRC-a3f9c2... , random on every run) instead of three backticks. With a fixed fence, a file containing three backticks closes its own block and the rest is read as instructions: on someone else's code that is an open door.

Structured output per provider:

Provider	Mechanism
Gemini	<code>responseMimeType: application/json + full responseSchema</code>
Azure OpenAI	Responses API (/openai/v1/responses): <code>text.format = json_object ;</code> on the classic dialect <code>response_format</code>
Anthropic	assistant-turn prefill with <code>{</code>

The prefill costs nothing and removes at the root the preambles and markdown fences that were the first cause of unparsable JSON on that provider.

Phase 4 — The model chain

`model_chain.py`. The model is not a configuration constant.

How much it should think. The *Reasoning effort* selector (Fast · Balanced · Deep) becomes `thinkingLevel` on Gemini, `thinking` with a budget on Anthropic, `reasoning_effort` on Azure. It is the lever that moves response time the most, far more than the choice of model. The chosen level is a **starting point**: if a model refuses it with a 400 (some will not go below `medium`), the app moves up one step, remembers the minimum for that model and retries — whoever picks "Fast" gets the fastest that model can do, not an error. An adaptation does not consume the attempts reserved for real failures, and the probe always stays at the lowest level.

The preferred model. The *Preferred model* menu in the sidebar is filled with the models discovery finds on the key; *Automatic* lets the chain decide, a specific entry puts that model first (`preferito` in `CatenaModelli`), the others as fallback, and the choice wins over memory.

Discovery. `GET /v1beta/models` (Gemini), `GET /v1/models` (Anthropic), `GET /openai/deployments` (Azure). `-lite`, `preview`, `exp`, `latest`, dated names, embedding/tts/vision/live are filtered out; Gemini requires version ≥ 2.0 . At most five models remain, ordered by **tier** then by version: with `Quality first` the chain starts from `pro/opus/gpt-5,o3` and descends; with `Speed first` you get the Nuvia rule (fast ones only). On Azure the hand-typed deployment always stays first, because only whoever created the resource knows the callable name. If discovery fails, a chain written in the code is used: it is a safety net, not a failure to show.

Probe. To each model, in order: "Answer with a single word: OK", ceiling **5 s per model** and **10 s overall**. The first that answers wins. A second attempt on the same model only on transient failures and only if at least another 5 s remain: otherwise the second chance is the next model, which is worth more. A 200 with empty text counts as alive (reasoning models can spend the test's token cap before writing).

Real call. Starts from the model that answered and goes **only downwards**, never back up: three attempts per model with growing waits.

Failure taxonomy, single for the three providers: `quota`, `modello`, `badkey`, `busy`, `timeout`, `rete`, `vuota`, `troncata`, `blocked`, `contesto`, `parametro`. Each provider translates its own HTTP dialect into these names. Three sets decide the behaviour: `RIPROVA` (same model), `CAMBIA` (next model), `NON_SCENDERE` (pointless to go on).

The 400 that is not a bad key. Before blaming the key, the body of the error is read: if it mentions schema, `temperature`, `max_completion_tokens` or thinking, the request is adapted, **the adaptation is remembered for that model**, and it retries. Next time it starts right. Same mechanism as Nuvia's `thinkingLevel`.

Wall-clock guard. Every request runs in a thread against a real deadline, because the `requests timeout` is per socket read: a server sending one byte every three seconds never trips it, and the five-second ceiling stops existing.

Memory. Good model 10 minutes, discovered list 6 hours, adaptations forever. On file or in RAM. Of the key only a truncated SHA-256 fingerprint is saved as the cache identity: two keys do not share data and the key never reaches the disk.

Messages. "Try again later" only if trying again can change something. Bad key, exhausted quota, full context each have their own message, in Italian and in English.

Transport is plain `requests`, not the three SDKs: the SDKs retry on their own *on top of* our fallback (the OpenAI client retries 429s by default, eating our ceiling's time) and hide the HTTP status the taxonomy uses to decide.

Phase 5 — Return and normalisation

`estrai_json` strips any fences, tries `json.loads`, then tries the subset between the first `{` and the last `}`, then calls `ripara_troncato`.

`ripara_troncato` walks the text tracking strings, escape sequences and the bracket stack, goes back to the last safe boundary and closes what was open. **The token cap is the most common way to lose a three-minute analysis:** before, the half-written last row threw away the two hundred perfect rows that came before it too.

`normalizza` does the rest: missing fields filled, similar field names recovered, enums straightened with a synonym table ("High", "critico", "Sev2"), lists turned into text (A; B), cells cut at 1,500 characters, empty rows dropped, duplicates removed on the declared keys, identifiers assigned **by the code** (the model restarts from 1 in every batch), Mermaid cleaned. Everything it had to fix goes into `contract_warnings`, visible in the interface: whoever validates must know how much to trust.

The Mermaid cleaning deserves a line: labels are put in quotes and cleaned of `( ) [ ] { } " ; |` with a hand-written scanner, not a regex, because the shapes nest (`A[Order (new)]`) and a regex catches the inner parenthesis leaving out the one that matters. The same scanner tells visible text from identifiers, and in the latter renames the JavaScript prototype properties (`toString`, `valueOf`, `constructor`...) that would otherwise kill the renderer. `end` and `subgraph` stay untouched: there the word has a meaning.

Consolidation. With more than one batch, a final call runs that receives **only the inventory**, no code: it produces one summary of the whole application and looks for links between different batches. Every name that comes back is checked against the inventory — in that call the model has no code in front of it, so nobody would contradict an invented name. If it fails, the per-batch summaries are kept and it is declared.

Completeness and continuation. The contract requires the last property to be `"complete": true`. `risposta_completa` deems an answer complete if the JSON parses in full and the tag is there, or — without the tag — if the provider did not flag it as cut. If it is not, `CatenaModelli.continua` asks **the same model** for the rest (on Anthropic the written part is the prefill and the model continues the same sentence; on Gemini and Azure it goes back as the previous turn), `unisci_continuazione` stitches it on trimming the overlap, and the check runs again. Two automatic rounds; then the four choices (same model, next model via `CatenaModelli.successivo` only if the first stops answering, keep the partial, discard). An automatic continuation that fails does not bring the run down: the batch stays incomplete with its cause. The batch state — prompt, text, model — lives in the session and not in the exported JSON, because the prompt contains the customer's source.

`unisci` adds up the batches on the same deduplication keys, concatenates the texts, keeps the most complete diagram (two `flowchart TD` glued together do not render) and rennumbers.

`merge_static_and_ai_results` passes **the parser's rows too** through the contract before merging: it is the only way for the tables to have the same columns.

Phase 6 — The diagrams

`diagrams.py` builds the four drawings from the structured data: `call_graph` from `dependencies`, `application map` from `application_mapping` (or `interfaces`), `data flow` from `data_flows` (or `data_objects`), `process flow` from `business_processes`. They are built **after** the merge with the static analysis, so the call graph also contains the dependencies found by the parser.

`arricchisci` chooses: if the model produced something substantial (≥ 3 lines) its version stays, otherwise the built one is shown. Both versions remain in the result and the interface has a switch, plus a button that redraws from the current tables — that is, after the SME's corrections. Provenance is written next to the drawing, **not** among the contract warnings: building the diagram from the data is the intended behaviour, and in the warnings panel it looked like a fault.

Guarantees by construction: unique ids always prefixed with `n_` — a bare id can coincide with a property every JavaScript object inherits (`toLocaleString`, `constructor`), and Mermaid, which keeps nodes in a plain object, believes the node already exists and dies with "Cannot set properties of undefined (setting 'order')"; labels without the characters that break Mermaid; a cap of 40 edges with a note on how many are left out; when cutting, `PROBABLE_CALL`s and low confidences go first.

Phase 7 — Rendering and export

The drawing configuration (theme, SVG labels instead of HTML, wrapping width) is written **inside** the diagram as a `%%{init: ...}%%` directive: a configuration file applies only to local drawing, while the external service receives only the code.

`mermaid_render.rendi` tries, in order: local `mmdc` (`@mermaid-js/mermaid-cli`, i.e. mermaid.js inside a headless Chromium), then `npx` if explicitly enabled, then `mermaid.ink` as fallback. `MERMAID_LOCAL_ONLY=1` forbids going out altogether. It handles `--no-sandbox` for containers with a temporary Puppeteer configuration file, honours `PUPPETEER_EXECUTABLE_PATH`, and caches on the code's hash (PDF and Word ask for the same four diagrams).

`exporter.py` prepares **one single description** of the document — cover, headings, prose, tables, diagrams — and renders it twice, as PDF (ReportLab, landscape A4, IBM Plex) and as Word (python-docx). Before, they were two parallel functions that had already stopped saying the same things.

Tables carry the provenance of every row (`parser` or `model`), confidence as dots and the expert's confirmation mark; risks come out ordered by severity and dependencies by reliability. Images honour **the proportions read from the image itself** (PNG or JPEG header, otherwise PIL); there is no fixed fallback size.

The document has eight sections: summary, business logic, architecture, data, risks, diagrams, **open questions and assumptions**, and in the appendix what the parser found.

State, cache and costs

Streamlit re-runs the whole script on every interaction. Consequently:

- the result lives in `st.session_state` and the tables' edits go back into it on every pass; the batch state (prompt, text written so far, model) sits in `st.session_state["lotti_stato"]`, separate from the result, because the prompt contains the customer's source and must not end up in the exported JSON;
- `analysis_signature` (file hashes + provider + depth + contract version) avoids re-running an analysis identical to the previous one;
- the `cache/` folder (visible, excluded from git, deletable) keeps the batches already analysed — key: file hashes, contract, provider, preference, reasoning, depth — and the chain's memory; only complete batches go in;
- PDF and Word are generated **on demand** and sit in `st.cache_data`: before, every tick in a box regenerated both, downloading eight images;
- the exported JSON also carries `_metadata` and who answered, and is loaded back with `carica_analisi_salvata`, which passes it through the contract like a model answer (ticks included) and rebuilds the diagrams.

Test suite

`python test_catena.py` — 227 cases, no network, no key, no cost. The provider is fake: you declare which models answer and how, and observe the behaviour. It covers discovery and ordering, fallback, selective retry, memory, messages in both languages, time ceilings, parameter adaptation, JSON repair, enums, deduplication, Mermaid, batches, image geometry and diagram construction.

Known limits

- No number says how much of the application has been captured**, and none can: it would require knowing the answer in advance. The indicators measure grounding (files cited, parser components and tables actually described, rows with evidence, confidence, unresolved calls). The **silent files** — those full of IF and CASE from which no rule came out — are the strongest signal the application can give about itself.
- Java, COBOL and RPG stay on heuristics**. In Python calls are read from the syntax tree and in SQL the tree confirms them; for the other languages a dedicated parser would be needed, which is a project of its own. Those rows stay marked and counted for what they are.
- Consolidation sees the inventory, not the code**. It can link two names already found, not discover what no batch saw. Batches are now formed along the dependencies, so it has much less to recover — but the gap is not closed.
- Keys sit in the sidebar**. Fine for a prototype; for several users they need environment variables or a secrets manager.
- Word uses Arial and Courier New**, not the application's fonts: they are the only ones that exist everywhere, and a delivery document must look the same to whoever opens it. The PDF has the fonts inside the file and uses IBM Plex.
- An unvalidated document is worth a draft**. The application produces a first reasoned reading; the signature is a person's.

Note on the contract

The JSON contract is at version **2.1**. An exported JSON is loaded back from the "*Or resume a saved analysis*" panel under step 2, with the expert's ticks and corrections; so are those saved with 2.0 — the missing `steps` field stays empty, and in old analyses the diagrams will not have the execution order of the steps.